TASKFLOW

PERSONAL TASK MANAGER

PHASE II CAPSTONE PROJECT DOCUMENTATION

Project Information	Details
Course	SEC042 – New Age Programming Languages Lab
Semester	V
Academic Year	2026–2027
Primary Programming Language	Go (Golang)
Platform	Web Application
Project Type	Individual Capstone Project

Prepared as Phase II project documentation

1. PROJECT OVERVIEW AND OBJECTIVES

1.1 Introduction

TaskFlow is a personal task management web application developed using Go (Golang) as part of the B.Tech Semester V capstone project for the SEC042 – New Age Programming Languages Lab.

The application aims to provide users with a simple and organised platform for managing daily tasks, tracking deadlines, assigning priorities and monitoring task progress.

The system follows a modular backend architecture and uses JSON-based file storage to maintain task information. Its frontend provides a dashboard and task management interface designed to make everyday task organisation easier. **1.2**

Project Objectives

- Develop a functional task management application using Go.
- Implement a modular software architecture.
- Understand HTTP server development using Go's standard library.
- Implement task creation, retrieval, updating and deletion.
- Store task information using JSON-based file persistence.
- Develop a user-friendly web interface.
- Understand communication between frontend and backend components.
- Apply software testing practices during application development.

1.3 Problem Statement

Managing multiple tasks, deadlines and priorities can become difficult without a centralised system. TaskFlow addresses this problem by providing a single application through which users can organise tasks, monitor their progress and maintain task-related information.

2. SYSTEM ARCHITECTURE AND DIRECTORY STRUCTURE

2.1 Architectural Overview

TaskFlow follows a layered architecture that separates the application's responsibilities into different components: Models, Handlers, Services, Storage and Presentation.

General request flow: Frontend → HTTP Handlers → Task Service → JSON Storage. The response travels back through the same layers to update the frontend.

2.2 Project Directory Structure

```
Go-Task-Manager/
■■■ main.go
■■■ models/
■   ■■■ task.go
■■■ handlers/
■   ■■■ task_handler.go
■■■ services/
■   ■■■ task_service.go
■■■ storage/
■   ■■■ json_storage.go
■■■ templates/
■   ■■■ index.html
■   ■■■ all_tasks.html
■   ■■■ empty_state.html
■   ■■■ in_progress.html
■■■ static/
■   ■■■ css/style.css
■   ■■■ js/app.js
■   ■■■ assets/
■■■ data/
■   ■■■ tasks.json
■■■ tests/
■   ■■■ task_test.go
■■■ docs/
■   ■■■ Phase2_Documentation.md
■■■ screenshots/
■■■ README.md
■■■ go.mod
■■■ .gitignore
■■■ LICENSE
```

Note: Confirm that every listed file exists in the submitted repository. Remove any entry that is not present.

2.3 Architectural Components

- **Models Layer:** Defines the application's data structures and domain types.
- **Handlers Layer:** Receives HTTP requests, processes incoming data and communicates with the service layer.
- **Services Layer:** Contains the business logic responsible for managing task operations.
- **Storage Layer:** Handles reading and writing task information to the JSON data file.
- **Presentation Layer:** Contains HTML templates, CSS styling, JavaScript functionality and frontend assets.

3. COMPONENT RESPONSIBILITIES

3.1 Main Application – main.go

The main.go file acts as the entry point of the application. Its responsibilities include initialising JSON storage, creating the task service, loading HTML templates, registering HTTP routes, configuring static file serving and starting the Go HTTP server. The application uses Go's net/http package to handle HTTP requests. The default server port is 8081.

3.2 Task Model – models/task.go

The task model defines the structure of individual tasks.

Attribute	Description
ID	Unique identifier of the task
Title	Name of the task
Description	Additional task details
Priority	Task priority level
Status	Current task progress
Category	Category assigned to the task
Attribute	**Description**
DueDate	Task deadline
CreatedAt	Task creation timestamp
UpdatedAt	Last modification timestamp

Priority levels: Low, Medium and High. **Supported statuses:** Pending, In Progress and Completed.

JSON tags are used to support data exchange between the application and JSON storage.

3.3 HTTP Handlers – handlers/task_handler.go

The handler component manages incoming HTTP requests and application views. Its responsibilities include serving the dashboard, All Tasks interface and Empty State interface; retrieving task information; processing task creation, update and deletion requests; and returning JSON responses with appropriate HTTP status codes.

3.4 Task Service – services/task_service.go

The service layer contains the application's task management logic.

- **GetAllTasks():** Retrieves all available tasks from the storage layer.
- **GetTaskByID():** Retrieves an individual task using its unique identifier.
- **CreateTask():** Creates a new task, generates a unique identifier, assigns timestamps and saves the updated task list.
- **UpdateTask():** Updates an existing task while preserving its original creation timestamp.
- **DeleteTask():** Removes a task from the stored task collection using its identifier.

The service layer also performs basic validation of task titles, priorities and statuses.

3.5 JSON Storage – storage/json_storage.go

The storage component implements JSON-based persistence. LoadTasks() reads task information from the configured JSON file and converts it into Go task structures. SaveTasks() converts task information into JSON format and writes it to the storage file.

The implementation uses encoding/json for JSON encoding and decoding, os for file operations, and sync.RWMutex for synchronisation during file access. Task information is stored in data/tasks.json.

4. USER INTERFACE DESIGN AND INTEGRATION

4.1 Design Direction

TaskFlow follows a minimal, modern and natural visual design approach. The interface uses warm neutral backgrounds and soft green accents to create a comfortable working environment.

Element	Colour	Hex Code
Background	Warm Ivory	#FAF7F2

Element	Colour	Hex Code
Typography	Warm Charcoal	#2C2C2A
Surfaces	Soft Cream	#F4EFE6
Primary Accent	Sage Green	#7A9A60
Secondary Accent	Muted Olive	#556B2F
Highlights	Terracotta	#D96B43
Cards	White	#FFFFFF

4.2 Google Stitch Integration

Google Stitch was used during the frontend design process to generate the initial application interfaces. The Stitch project was connected through Stitch MCP.

Three screens were successfully imported:

- **Dashboard Overview – templates/index.html:** Provides the primary application dashboard, including dashboard metrics, task cards, task management interface, task creation modal and navigation elements.
- **All Tasks Workspace – templates/all_tasks.html:** Provides a workspace for viewing and organising tasks through a table-based layout.
- **Empty State – templates/empty_state.html:** Provides an onboarding interface for situations where no tasks are available.

The imported interfaces serve as the foundation for the application's frontend. Additional screen designs were not imported due to available generation limits.

4.3 Frontend Technologies

- HTML for page structure.
- CSS for styling.
- JavaScript for user interactions.
- Tailwind CSS utilities where included in the imported interface.
- Static assets for visual elements.

The frontend is served through Go's HTML template and static file handling capabilities.

5. FUNCTIONAL IMPLEMENTATION

5.1 Task Management Operations

TaskFlow is designed around the four fundamental CRUD operations.

Operation	Description
Create	Adds a new task with its details
Read	Retrieves and displays stored tasks
Update	Modifies existing task information
Delete	Removes an existing task

The backend service and HTTP handler functions for these operations have been implemented.

5.2 Task Attributes

Each task supports task title, description, due date, priority, status and category. These attributes help users organise tasks according to their requirements.

5.3 API Design

The application exposes HTTP endpoints for task management.

HTTP Method	Endpoint	Purpose
GET	/api/tasks	Retrieve all tasks
GET	/api/tasks/{id}	Retrieve a specific task
POST	/api/tasks	Create a new task
PUT	/api/tasks/{id}	Update an existing task
DELETE	/api/tasks/{id}	Delete a task

The API uses JSON for request and response data.

5.4 Data Persistence

Task information is maintained in a local JSON file. The storage layer loads existing task information when requested and saves modifications to the configured data file. This approach provides lightweight persistence without requiring an external database management system.

6. TECHNOLOGIES AND TOOLS USED

Technology / Tool	Purpose
Go (Golang)	Backend programming language
HTML5	Frontend structure
CSS3	Interface styling
JavaScript	Frontend interactions
Tailwind CSS	Utility-based styling where included
JSON	Data persistence
Go net/http	HTTP server and routing
html/template	HTML template rendering
encoding/json	JSON encoding and decoding
Google Stitch	Initial UI design
Stitch MCP	UI import integration
Visual Studio Code	Development environment
Git	Version control
GitHub	Repository hosting

7. TESTING AND VERIFICATION

7.1 Testing Approach

The project uses Go's built-in testing framework. Testing was performed to verify package compilation and the existing test suite.

Command executed:

```
go test ./...
```

7.2 Observed Test Results

The existing test execution completed successfully. The observed output included: `ok`

```
go-task-manager/tests
```

The existing tests cover portions of the application's model, storage, service and handler functionality. The test suite passed without reported failures during the recorded test run.

7.3 Application Execution Verification

The Go HTTP server was successfully started using: `go`

```
run .
```

The application was accessible locally through: `http://localhost:8081/`

- Successful Go server startup.
- Successful rendering of the dashboard interface.
- Display of the task creation modal.
- Display of the imported application interface.

7.4 Pending Verification

The following activities require further integration testing:

- Verification of task creation through the frontend.
- Verification of JSON persistence after task creation.
- Verification of task updating through the frontend.
- Verification of task deletion through the frontend.
- Complete frontend-backend integration testing.

The successful execution of the existing Go tests does not independently establish that every frontend workflow is functioning correctly.

8. APPLICATION EXECUTION INSTRUCTIONS

8.1 Prerequisites

- Go programming language installed.
- Visual Studio Code or another code editor.
- A modern web browser.

8.2 Running the Application

Step 1: Open the project directory in Visual Studio Code.

Step 2: Open the integrated terminal.

Step 3: Execute the application: `go`

```
run .
```

Step 4: Wait for the server startup message.

Step 5: Open a browser and navigate to: `http://localhost:8081/`

The TaskFlow dashboard should appear.

8.3 Data Storage Location

Task information is stored in data/tasks.json. This file contains the application's JSON-based task records.

9. PROJECT SCREENSHOTS

The following screenshots are intended to demonstrate the application's user interface and development progress. Insert genuine screenshots captured from the running application in the spaces provided.

Screenshot 1 – Dashboard Overview

[Insert corresponding application screenshot here]

Screenshot 2 – Create New Task Interface

[Insert corresponding application screenshot here]

Screenshot 3 – All Tasks Workspace

[Insert corresponding application screenshot here]

Screenshot 4 – Empty State Interface

[Insert corresponding application screenshot here]

10. CONCLUSION AND FUTURE SCOPE

10.1 Conclusion

TaskFlow demonstrates the development of a personal task management web application using Go and modern frontend technologies.

The project applies modular programming principles by separating models, HTTP handlers, business logic and data storage into independent components.

The application uses Go's standard library for HTTP server development, HTML template rendering and JSON processing. The project also provides practical experience in frontend design, backend development, data persistence, API design and software testing.

10.2 Future Scope

- Complete frontend-backend integration.
- Enhanced task filtering and sorting.
- Improved task search functionality.
- Additional dashboard analytics.

- User authentication and account management.
- Database integration for larger datasets.
- Improved validation and error handling.
- Deployment to a cloud hosting platform.

11. PROJECT REPOSITORY

Project Name: TaskFlow – Personal Task Manager

GitHub Repository: https://github.com/Argy2006/Capstone_NAPL

Primary Language: Go (Golang)

Course: SEC042 – New Age Programming Languages Lab

Project Type: Individual Capstone Project

End of Documentation